

Feature Engineering

- So far we have performed feature analysis
 - Data frame quick checks
 - Categorical column analysis
 - Bar charts , pie charts
 - Numerical column analysis
 - Histograms, box plots , outliers
 - Bi-multi variate analysis
 - correlation etc
- Now we need to learn Feature Engineering
 - we will create new columns for better ML models
 - we will perform the **Encoding**, which is convert categorical to Numerical data
 - we will do data **transformations**
 - we will perform **Scaling: Standardization and Normalization**
 - we will perform the **missing value analysis**
- Simply feature engineering means our data will be modified

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
cat=visa_df.select_dtypes(include='object').columns
num=visa_df.select_dtypes(exclude='object').columns
```

```
In [4]: cat
```

```
Out[4]: Index(['case_id', 'continent', 'education_of_employee', 'has_job_experience',
              'requires_job_training', 'region_of_employment', 'unit_of_wage',
              'full_time_position', 'case_status'],
              dtype='object')
```

case_status

```
In [7]: # how many unique labels
visa_df['case_status'].unique()
```

```
Out[7]: array(['Denied', 'Certified'], dtype=object)
```

```
In [9]: # create a mapper
d={'Certified':0, 'Denied':1}
```

```
In [15]: # Overwrite in same column
visa_df['case_status']=visa_df['case_status'].map(d)
# 1 0 1 1 0
```

```
In [17]: visa_df
```

```
Out[17]:
```

	case_id	continent	education_of_employee	has_job_experience	requires_job_1
0	EZYV01	Asia	High School		N
1	EZYV02	Asia	Master's		Y
2	EZYV03	Asia	Bachelor's		N
3	EZYV04	Asia	Bachelor's		N
4	EZYV05	Africa	Master's		Y
...	...	...	...	...	...
25475	EZYV25476	Asia	Bachelor's		Y
25476	EZYV25477	Asia	High School		Y
25477	EZYV25478	Asia	Master's		Y
25478	EZYV25479	Asia	Master's		Y
25479	EZYV25480	Asia	Bachelor's		Y

25480 rows × 12 columns



```
In [29]: labels=sorted(visa_df['continent'].unique())
d={}
for i,j in enumerate(labels):
    #print(j,i)
    d[j]=i
d
```

```
Out[29]: {'Africa': 0,
'Asia': 1,
'Europe': 2,
'North America': 3,
'Oceania': 4,
'South America': 5}
```

```
In [33]: {i:j for j,i in enumerate(labels)}
```

```
Out[33]: {'Africa': 0,
'Asia': 1,
'Europe': 2,
'North America': 3,
'Oceania': 4,
'South America': 5}
```

```
In [39]: labels=sorted(visa_df['continent'].unique())
d={i:j for j,i in enumerate(labels)}
visa_df['continent']=visa_df['continent'].map(d)
```

```
In [41]: visa_df
```

```
Out[41]:
```

	case_id	continent	education_of_employee	has_job_experience	requires_job_1
0	EZYV01	1	High School	N	
1	EZYV02	1	Master's	Y	
2	EZYV03	1	Bachelor's	N	
3	EZYV04	1	Bachelor's	N	
4	EZYV05	0	Master's	Y	
...	...	...	...	...	...
25475	EZYV25476	1	Bachelor's	Y	
25476	EZYV25477	1	High School	Y	
25477	EZYV25478	1	Master's	Y	
25478	EZYV25479	1	Master's	Y	
25479	EZYV25480	1	Bachelor's	Y	

25480 rows × 12 columns



```
In [43]: # read the data again
visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
cat=visa_df.select_dtypes(include='object').columns
num=visa_df.select_dtypes(exclude='object').columns

for col in cat[1:]:
    labels=sorted(visa_df[col].unique())
    d={i:j for j,i in enumerate(labels)}
    visa_df[col]=visa_df[col].map(d)
```

```
In [45]: visa_df
```

Out[45]:

	case_id	continent	education_of_employee	has_job_experience	requires_job_1
0	EZYV01	1	2	0	
1	EZYV02	1	3	1	
2	EZYV03	1	0	0	
3	EZYV04	1	0	0	
4	EZYV05	0	3	1	
...	...	...	...	...	...
25475	EZYV25476	1	0	1	
25476	EZYV25477	1	2	1	
25477	EZYV25478	1	3	1	
25478	EZYV25479	1	3	1	
25479	EZYV25480	1	0	1	

25480 rows × 12 columns



In []:

- last **class** we performed the encoding
- we created a dictionary , we map to original data
- overwrite **in** the same column
- the same we can develop by using sickit-learn library
- installation name : ****sickit-learn****
- **import** name : ****sklearn****
- Library: ****sklearn****
 - modules : ****preprocesing****
 - **class** : ****LabelEncoder****
 - method: ****fit_transform****

In [7]:

```
import sklearn
from sklearn import preprocessing
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
le.fit_transform
```

In [3]: sklearn

Out[3]: <module 'sklearn' from 'E:\\Anaconda\\Lib\\site-packages\\sklearn__init__.py'>

In []:

```
# import the class
# save the class
```

```
# apply fit transform
```

fit vs fit_transform

- if we are finding something then go for **fit**
- if we are finding some new values and using those values
- to modify the data then go for **fit + transform = fit_transform**
- we first developed a dictionary : **fit**
- we used the dictionary and change the data cat to num: **transform**

```
In [11]: # import the packages
# read the data
# cat and num

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
cat=visa_df.select_dtypes(include='object').columns
num=visa_df.select_dtypes(exclude='object').columns
```

```
In [17]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
visa_df['case_status']=le.fit_transform(visa_df['case_status'])
```

```
In [19]: visa_df
```

Out[19]:

	case_id	continent	education_of_employee	has_job_experience	requires_job_1
0	EZYV01	Asia	High School	N	
1	EZYV02	Asia	Master's	Y	
2	EZYV03	Asia	Bachelor's	N	
3	EZYV04	Asia	Bachelor's	N	
4	EZYV05	Africa	Master's	Y	
...	...	...	...	...	...
25475	EZYV25476	Asia	Bachelor's	Y	
25476	EZYV25477	Asia	High School	Y	
25477	EZYV25478	Asia	Master's	Y	
25478	EZYV25479	Asia	Master's	Y	
25479	EZYV25480	Asia	Bachelor's	Y	

25480 rows × 12 columns



```
In [ ]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
visa_df['case_status']=le.fit_transform(visa_df['case_status'])
#####
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
visa_df['continent']=le.fit_transform(visa_df['continent'])
#####
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
visa_df[i]=le.fit_transform(visa_df[i])
```

```
In [23]: # Read the data again
visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
for i in cat[1:]:
    visa_df[i]=le.fit_transform(visa_df[i])
```

```
In [25]: visa_df
```

```
Out[25]:
```

	case_id	continent	education_of_employee	has_job_experience	requires_job_1
0	EZYV01	1	2	0	
1	EZYV02	1	3	1	
2	EZYV03	1	0	0	
3	EZYV04	1	0	0	
4	EZYV05	0	3	1	
...	...	...	...	...	...
25475	EZYV25476	1	0	1	
25476	EZYV25477	1	2	1	
25477	EZYV25478	1	3	1	
25478	EZYV25479	1	3	1	
25479	EZYV25480	1	0	1	

25480 rows × 12 columns



explore about inverse transform in the Lab

```
In [28]: # again read the data
```

```
In [30]: visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
```

```
In [34]: data=le.fit_transform(visa_df['education_of_employee'])
```

```
In [36]: data[:5]
```

```
Out[36]: array([2, 3, 0, 0, 3])
```

```
In [44]: le.inverse_transform(data)[:5]
```

```
Out[44]: array(['High School', 'Master's', 'Bachelor's', 'Bachelor's', 'Master's'],
dtype=object)
```

```
In [42]: visa_df['education_of_employee'].unique()
```

```
Out[42]: array(['High School', 'Master's', 'Bachelor's', 'Doctorate'], dtype=object)
```

```
In [ ]: 'High School': 2
'Master's': 3
'Bachelor's': 0
'Doctorate': 1
#####
# ordinal data
# 'High School' < "Bachelor's" < "Master's" < "Doctorate"
#####
```

```
# 3>2>1>0
# "Master's">'High School'>'Doctorate'>"Bachelor's"

# flop === avg === hit == sh -- bb= is

# ramesh suresh sathis mahesh anish akash : nominal
```

- LabelEncoder use only for **Nominal level data**
- 90% target columns use only **LabelEncoder**
- for ordinal level data will go for **OrdinalEncoder**
- sklearn
 - preprocessing
 - OrdinalEncoder
 - fit_transform

```
In [50]: visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
visa_df['education_of_employee'].unique()
order=['High School',"Bachelor's","Master's",'Doctorate']
```

```
In [54]: from sklearn.preprocessing import OrdinalEncoder
oe=OrdinalEncoder(categories=order)
oe.fit_transform(visa_df['education_of_employee'])
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[54], line 3
      1 from sklearn.preprocessing import OrdinalEncoder
      2 oe=OrdinalEncoder(categories=order)
----> 3 oe.fit_transform(visa_df['education_of_employee'])

File E:\Anaconda\Lib\site-packages\sklearn\utils\_set_output.py:316, in _wrap_method_output.<locals>.wrapped(self, X, *args, **kwargs)
    314 @wraps(f)
    315 def wrapped(self, X, *args, **kwargs):
--> 316     data_to_wrap = f(self, X, *args, **kwargs)
    317     if isinstance(data_to_wrap, tuple):
    318         # only wrap the first output for cross decomposition
    319         return_tuple = (
    320             _wrap_data_with_container(method, data_to_wrap[0], X, self),
    321             *data_to_wrap[1:],
    322         )

File E:\Anaconda\Lib\site-packages\sklearn\base.py:907, in TransformerMixin.fit_transform(self, X, y, **fit_params)
    892         warnings.warn(
    893             (
    894                 f"This object ({self.__class__.__name__}) has a `transform`
m` "
    (...))
    902         UserWarning,
    903     )
    905 if y is None:
    906     # fit method of arity 1 (unsupervised transformation)
--> 907     return self.fit(X, **fit_params).transform(X)
    908 else:
    909     # fit method of arity 2 (supervised transformation)
    910     return self.fit(X, y, **fit_params).transform(X)

File E:\Anaconda\Lib\site-packages\sklearn\base.py:1336, in _fit_context.<locals>
>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
    1329     estimator._validate_params()
    1331 with config_context(
    1332     skip_parameter_validation=(
    1333         prefer_skip_nested_validation or global_skip_validation
    1334     )
    1335 ):
-> 1336     return fit_method(estimator, *args, **kwargs)

File E:\Anaconda\Lib\site-packages\sklearn\preprocessing\_encoders.py:1516, in OrdinalEncoder.fit(self, X, y)
    1509     raise TypeError(
    1510         "unknown_value should only be set when "
    1511         "handle_unknown is 'use_encoded_value', "
    1512         f"got {self.unknown_value}."
    1513     )
    1515 # `_fit` will only raise an error when `self.handle_unknown="error"`
-> 1516 fit_results = self._fit(
    1517     X,
    1518     handle_unknown=self.handle_unknown,
    1519     ensure_all_finite="allow-nan",
    1520     return_and_ignore_missing_for_infrequent=True,
    1521 )
    1522 self._missing_indices = fit_results["missing_indices"]

```

```

1524 cardinalities = [len(categories) for categories in self.categories_]

File E:\Anaconda\Lib\site-packages\sklearn\preprocessing\_encoders.py:86, in _BaseEncoder._fit(self, X, handle_unknown, ensure_all_finite, return_counts, return_and_ignore_missing_for_infrequent)
    84 self._check_infrequent_enabled()
    85 validate_data(self, X=X, reset=True, skip_check_array=True)
--> 86 X_list, n_samples, n_features = self._check_X(
    87     X, ensure_all_finite=ensure_all_finite
    88 )
    89 self.n_features_in_ = n_features
    91 if self.categories != "auto":

File E:\Anaconda\Lib\site-packages\sklearn\preprocessing\_encoders.py:53, in _BaseEncoder._check_X(self, X, ensure_all_finite)
    40 """
    41 Perform custom check_array:
    42 - convert list of strings to object dtype
    (...)
    49
    50 """
    51 if not (hasattr(X, "iloc") and getattr(X, "ndim", 0) == 2):
    52     # if not a dataframe, do normal check_array validation
--> 53     X_temp = check_array(X, dtype=None, ensure_all_finite=ensure_all_finite)
    54     if not hasattr(X, "dtype") and np.issubdtype(X_temp.dtype, np.str_):
    55         X = check_array(X, dtype=object, ensure_all_finite=ensure_all_finite)

File E:\Anaconda\Lib\site-packages\sklearn\utils\validation.py:1060, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)
   1053     else:
   1054         msg = (
   1055             f"Expected 2D array, got 1D array instead:\narray={array}.\n"
   1056             "Reshape your data either using array.reshape(-1, 1) if "
   1057             "your data has a single feature or array.reshape(1, -1) "
   1058             "if it contains a single sample."
   1059         )
-> 1060     raise ValueError(msg)
   1062 if dtype_numeric and hasattr(array.dtype, "kind") and array.dtype.kind in "USV":
   1063     raise ValueError(
   1064         "dtype='numeric' is not compatible with arrays of bytes/strings."
   1065         "Convert your data to numeric values explicitly instead."
   1066     )

ValueError: Expected a 2-dimensional container but got <class 'pandas.core.series.Series'> instead. Pass a DataFrame containing a single row (i.e. single sample) or a single column (i.e. single feature) instead.

```

```

In [58]: visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
visa_df['education_of_employee'].unique()
order=[['High School',"Bachelor's","Master's",'Doctorate']]

from sklearn.preprocessing import OrdinalEncoder
oe=OrdinalEncoder(categories=order)
oe.fit_transform(visa_df[['education_of_employee']])

```

```
Out[58]: array([[0.],
                [2.],
                [1.],
                ...,
                [2.],
                [2.],
                [1.]])
```

One Hot Encoder

- One hot means at a time only one value **ON**
- Other will be **OFF**
- ON means one value become **True**
- OFF means one value become **False**
- True means one value become **1**
- False means one value become **0**
- First we need to know how many unique labels are there
- for example **case_status** has two unique labels
 - Certified
 - Denied
- So one hot encoder creates two new columns
 - case_status_certified
 - case_status_denied

Case_Status	Certified	Denied
Certified	1	0
Denied	0	1
Certified	1	0
Denied	0	1

|Case_Status|Certified|Denied|

Header 1	Header 2
Row 1, Col 1	Row 1, Col 2
Row 2, Col 1	Row 2, Col 2

pd.get_dummies

